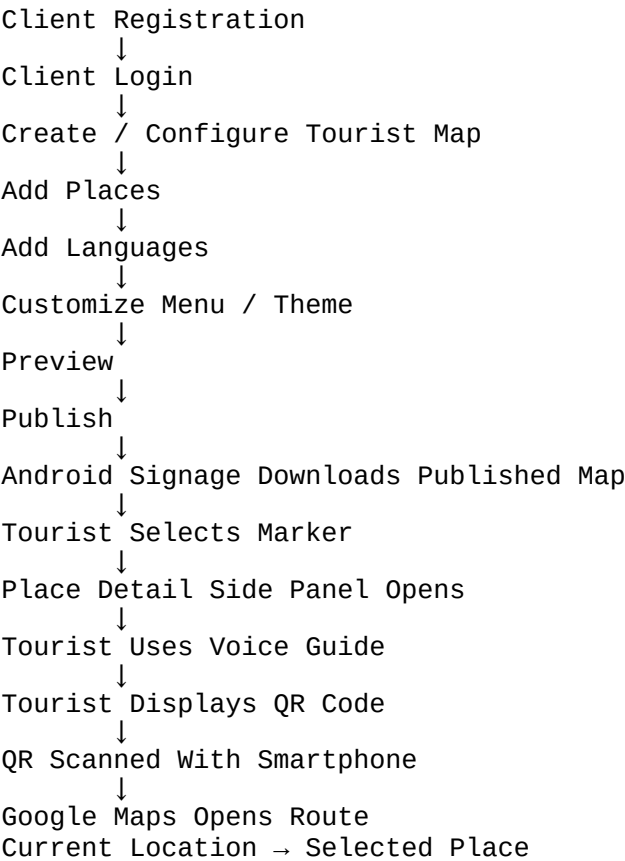


Tourist Map System — Stage 1 Blueprint

1. Stage 1 Goal

Stage 1 delivers the first complete working version of the Tourist Map System.

The full user flow is:



Stage 1 does **not** include the full Super Admin Store system yet.

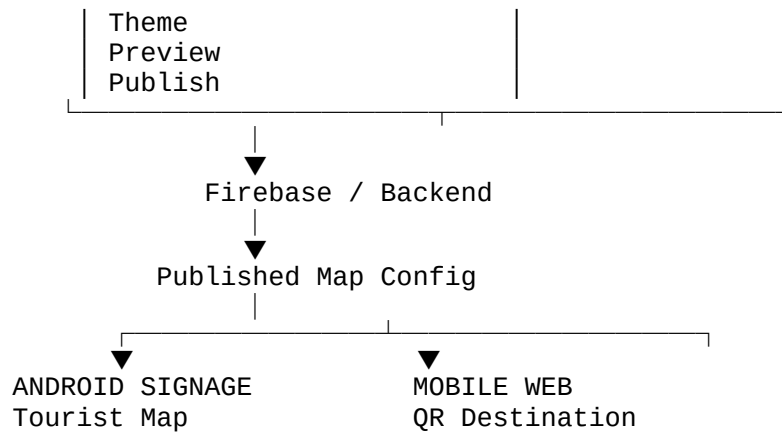
However, the architecture must already support future Super Admin features.

2. Stage 1 Main Systems

Stage 1 has three main applications.

STAGE 1

CLIENT ADMIN WEB	
Register	
Login	
Configure Map	
Add Places	
Languages	
Menu	



3. Stage 1 Technology

Client Admin

Next.js
TypeScript
React

Recommended UI:

Responsive desktop-first admin dashboard

The Client Admin is primarily intended for PC/tablet use.

Android Tourist Application

Kotlin
Jetpack Compose

Purpose:

Fullscreen Android signage application

Recommended orientation:

Landscape

Firebase

Use:

Firebase Authentication
Cloud Firestore
Firebase Storage

Optional backend:

Firebase Cloud Functions

or later:

Cloud Run

4. Stage 1 Roles

Stage 1 initially requires:

CLIENT_ADMIN
TOURIST

Tourist:

No login

Database should nevertheless support future:

SUPER_ADMIN
CLIENT_EDITOR

5. Multi-Tenant Foundation

Every client receives:

customerId

Every map receives:

mapId

Example:

customerId:
cust_001

mapId:
map_kyoto_001

Every client-owned resource must reference the owner.

Example:

customerId
mapId

A client can never access another client's data.

6. Stage 1 Repository Structure

Recommended monorepo:

```
tourist-map-system/  
├── apps/  
│   ├── admin-web/  
│   ├── mobile-web/  
│   └── android-signage/  
├── packages/  
│   ├── shared-types/  
│   ├── validation/  
│   ├── map-schema/  
│   └── localization/  
├── firebase/  
│   ├── firestore.rules  
│   ├── storage.rules  
│   ├── firestore.indexes.json  
│   └── functions/  
├── docs/  
└── README.md
```

The Android application can remain a separate Gradle project inside:

apps/android-signage

7. Development Environments

Use separate Firebase environments.

Development
Staging
Production

Example:

```
tourist-map-dev  
tourist-map-staging  
tourist-map-prod
```

Never use production Firebase while developing features.

8. Stage 1A — Foundation

Goal

Create the base platform and authentication system.

Deliverables:

Next.js Client Admin
Firebase project
Firebase Authentication
Firestore
Firebase Storage
Android base project
Shared data model
Client registration
Client login
Client logout
customerId generation
mapId generation

9. Client Registration

Registration page:

Create Your Tourist Map Account

Company Name
Contact Name
Email Address
Password
Confirm Password

[Register]

Optional Stage 1 fields:

Company Phone
Website
Client Type

Client Type examples:

Railway
Hotel
Municipality
Tourism Office
Shopping Facility
Other

10. Registration Workflow

Client submits registration

↓
Validate form
↓
Create Firebase Authentication account
↓
Generate customerId
↓
Create customer record
↓
Create user record
↓
Create first default map
↓
Generate mapId
↓
Redirect to Dashboard

If any critical operation fails:

Registration must not create partially usable customer data

Backend transaction / cleanup should be used.

11. User Record

Example:

users/{uid}

uid
customerId
role
displayName
email
status
createdAt
updatedAt

Role:

CLIENT_ADMIN

Status:

ACTIVE
DISABLED

12. Customer Record

customers/{customerId}

customerId
companyName
contactName
email
phone

website
clientType
status
createdAt
updatedAt

13. Map Record

maps/{mapId}

mapId
customerId
name
description

status

defaultLanguage
enabledLanguages

mapProvider
mapStyle

centerLatitude
centerLongitude
defaultZoom

rangeMode

themeId

draftVersion
publishedVersion

createdAt
updatedAt

Status:

DRAFT
PUBLISHED
ARCHIVED

14. Stage 1B — Client Dashboard

After login:

Tourist Map Admin		
Dashboard	Welcome	
Map Settings	Map Status	
Places		

Categories	Draft	
Languages		
Menu	Places: 12	
Appearance	Languages: 3	
Preview	[Preview]	
Publish	[Publish]	
Account		

15. Dashboard Summary

Show:

Map Name
 Published Status
 Number of Places
 Number of Categories
 Enabled Languages
 Current Published Version
 Last Published Time

Buttons:

Preview Map
 Publish
 Edit Map

16. Map Settings

Client can configure:

Map Name

Map Provider
 Google Maps

Map Style
 Road
 Satellite
 Hybrid

Map Center

Default Zoom

Map Range Mode
 Bounded
 Unbounded

Stage 1 should start with:

Google Maps

but the database architecture must support future Mapbox.

17. Select Map Center

Client can:

Search address

or:

Click position on map

Store:

centerLatitude
centerLongitude

18. Map Range

Stage 1 supports:

BOUNDED
UNBOUNDED

Bounded:

North
South
East
West

Admin should provide a visual rectangle editor later.

For first implementation, bounding box values can be generated from map interaction.

19. Stage 1C — Categories

Default categories created with map:

Sightseeing
Gourmet
Shopping
Events

Optional:

All

All can be a virtual category rather than stored.

20. Category Record

maps/{mapId}/categories/{categoryId}

categoryId

mapId

customerId

name

icon

displayOrder

enabled

translations

createdAt

updatedAt

21. Client Category Management

Client can:

Add Category

Rename

Enable

Disable

Change Icon

Reorder

Delete

Deletion should fail or warn if places still use that category.

22. Stage 1D — Places

Places are the core tourist content.

Admin page:

Places

[+ Add Place]

Search...

Image	Name	Category	Status
	Kyoto Tower	Sightseeing	Active
	Yodobashi	Shopping	Active
	Restaurant A	Gourmet	Draft

23. Add Place Workflow

Client selects:

+ Add Place

Then chooses:

Search Google Places
or
Create Custom Place

24. Google Places Import

Workflow:

Search:
ヨドバシカメラ 京都
↓
Google Places API
↓
Search Results
↓
Client selects location
↓
Import information
↓
Client customizes
↓
Save

Import:

Google Place ID
Name
Address
Latitude
Longitude
Main Photo
Rating
Website
Phone
Opening Hours

Only legally/permitted fields should be persisted according to provider rules.

25. Custom Place

Client enters:

Place Name
Category

Latitude
Longitude

Main Image

Description

Address

Website

Phone

Opening Hours

Coordinates can be chosen from an admin map.

26. Place Record

maps/{mapId}/places/{placeId}

placeId

customerId

mapId

source

googlePlaceId

categoryId

latitude

longitude

mainImageUrl

gallery

address

phone

website

rating

openingHours

markerType

markerImageUrl

enabled

translations

voiceGuide

createdAt

updatedAt

27. Place Source

GOOGLE_PLACES

CUSTOM

28. Marker Types

Stage 1:

PHOTO
DEFAULT_PIN
CUSTOM_IMAGE

Recommended default:

PHOTO

29. Marker Image

For Google Place:

Place Photo

For Custom Place:

Client Uploaded Image

Fallback:

Category Icon

30. Stage 1E — Languages

Client page:

Languages

Default Language:
English

Enabled:

✓ English
✓ 日本語
✓ 中文
한국어
Français

Stage 1 recommended initial languages:

English
Japanese
Simplified Chinese
Korean

Even if the first demo uses only three.

31. Translation Model

For every place:

translations:

en:

title
shortDescription
description

ja:

title
shortDescription
description

zh-CN:

title
shortDescription
description

Same concept for:

Categories

Menu

Events later

32. Translation Editor

Place editor:

[English] [日本語] [中文]

Title

Short Description

Description

[Save]

Missing translation should be visually indicated.

Example:

English	Complete
Japanese	Complete
Chinese	Missing

33. Fallback Language

If translation is missing:

Selected Language



Translation exists?

Yes

No



Display

Default Language

34. Stage 1F — Client Menu Builder

Admin page:

Menu

☰	Sightseeing	Enabled
☰	Gourmet	Enabled
☰	Shopping	Enabled
☰	Events	Enabled
☰	Audio Guide	Enabled
☰	Search	Enabled

Client can:

Enable
Disable
Rename
Reorder
Change icon

35. Menu Record

maps/{mapId}/menu/{menuItemId}

menuItemId
type
linkedCategoryId

enabled
displayOrder
icon

translations

Types:

CATEGORY
SEARCH
AUDIO_GUIDE

Future:

LIVE_CAMERA
WIDGET
CUSTOM

36. Stage 1G — Appearance

Client can configure:

Logo
Map Title
Primary Theme Color
Secondary Theme Color
Bottom Menu Style
Marker Style

Do not allow arbitrary CSS.

Use controlled theme properties.

37. Theme Record

For Stage 1, theme can live inside map settings:

```
theme:  
  primaryColor  
  secondaryColor  
  surfaceColor  
  textColor  
  logoUrl
```

Later it can reference Store themes.

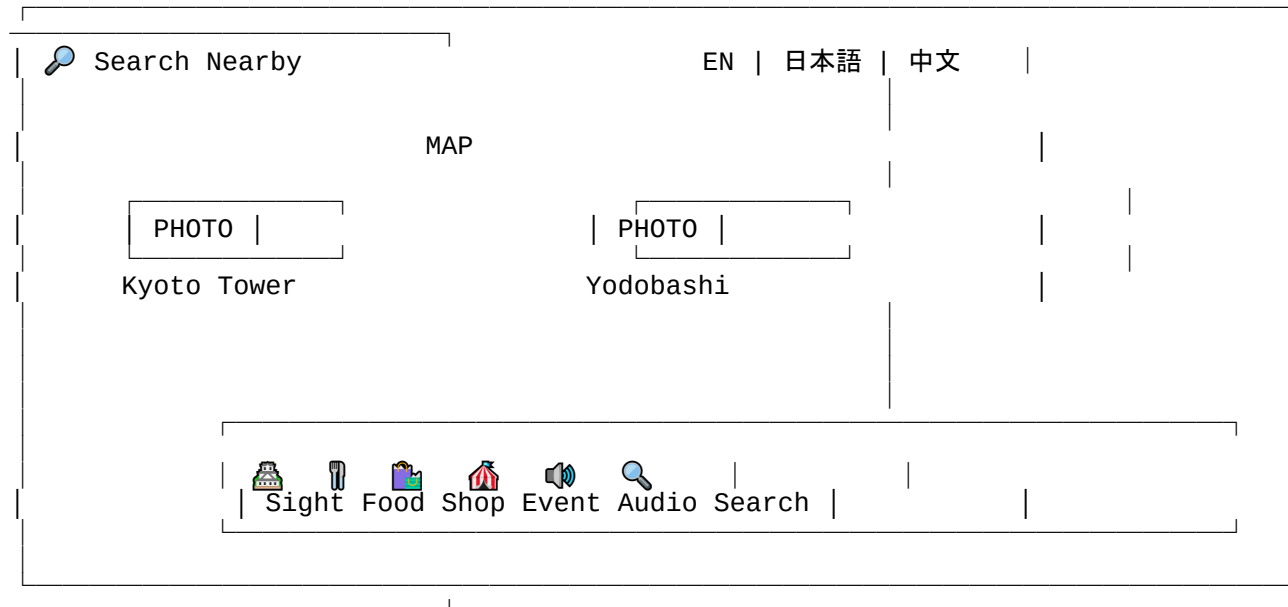
38. Stage 1H — Android Tourist Map

Android startup:

```
App starts  
↓  
Load device configuration  
↓  
Identify mapId  
↓  
Load local published map cache  
↓  
Check server for new version  
↓  
Download if newer  
↓  
Render Tourist Map
```

39. Android Main Screen

Target design:



40. Android Map Rules

Recommended signage defaults:

Rotation disabled
Tilt disabled
Limited zoom
Limited pan

Map should automatically return to default state after inactivity.

41. Category Filter Behavior

Tourist taps:

Gourmet

Then:

Only Gourmet markers shown

Tap:

All

Then:

All enabled places shown

42. Marker Tap Behavior

Tourist taps marker
↓
selectedPlaceId set
↓
Map centers selected marker
↓
Map slightly zooms
↓
Left detail pane slides open

43. Stage 1 Place Detail Side Panel

Target UI:



Recommended width:

38%

Map remains:

62%

44. Panel Content Priority

Stage 1 displays:

Image
Title
Description
Audio Guide
QR Button
Address
Opening Hours

Optional:

Rating
Phone
Website

45. Detail Panel Close Rules

Close when:

X pressed
Empty map area pressed
Another marker selected
Idle timeout reached

46. Stage 1I — Voice Guide

Button:

 Audio Guide

Behavior:

Current Language
↓
Load selected place description
↓
Text-to-Speech
↓
Play through signage speakers

47. Voice State

Android should support:

Play
Pause
Stop

Replay

If another marker is selected:

Stop current voice automatically

If language changes:

Stop current voice

48. Stage 1 TTS Architecture

Create abstraction:

VoiceGuideProvider

Initial provider:

ANDROID_TTS

Future:

GOOGLE_CLOUD_TTS

AZURE_TTS

AWS_POLLY

UPLOADED_AUDIO

AI_VOICE

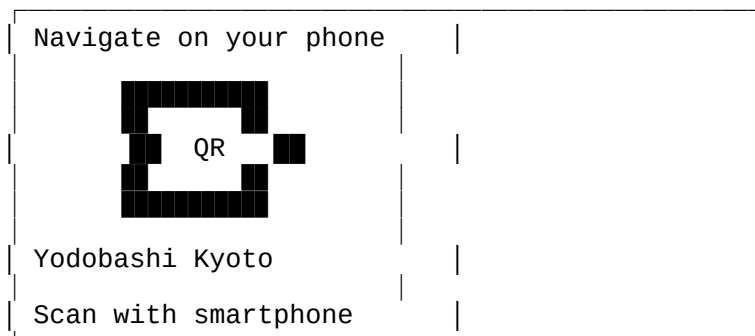
49. Stage 1J — QR Navigation

Tourist selects:

Show QR

Open QR modal inside side panel or centered overlay.

Example:



50. QR URL Format

Use:

`https://tourist.example.com/m/{mapId}/p/{placeId}`

Example:

`https://tourist.example.com/m/map_kyoto_001/p/place_yodobashi`

Do not directly encode only a Google Maps URL.

51. Stage 1K — Mobile Destination Page

When scanned:

QR



Mobile Web Page

Page:

[Image]		
Yodobashi Kyoto		
Short Description		
[Open Directions]		

52. Directions Behavior

When tourist presses:

Open Directions

Use smartphone navigation:

Current Location



Selected Place

Initial destination provider:

Google Maps

Future:

Apple Maps

Mapbox

Other navigation

53. Stage 1L — Search

Android search should search client-published places only.

Example:

Search:
sushi

Result:

Tokyo Sushi Chef
Seafood Paradise
Sushi Restaurant A

Touch result:

Center location
Open detail panel

54. Search Fields

Search against:

Place title
Short description
Category
Keywords

Use current selected language.

55. Stage 1M — Preview

Client must be able to preview before publishing.

Client Admin:

[Preview]

Preview should simulate:

Android landscape layout
Markers
Bottom menu
Language
Detail panel
Theme

Preview uses:

Draft Data

Android uses:

Published Data

56. Stage 1N — Publish System

Publish button:

[Publish Map]

Workflow:

```
Client presses Publish
      ↓
Validate Draft
      ↓
Ensure required data exists
      ↓
Generate version number
      ↓
Create immutable published snapshot
      ↓
Set currentPublishedVersion
      ↓
Notify / allow Android sync
```

57. Publish Validation

Block publishing if:

- No map center
- No enabled language
- Default language disabled
- Invalid map bounds
- Invalid place coordinates
- Missing required client ownership
- Invalid menu configuration

Warnings can be used for:

- Missing optional translation
- Missing place image
- Missing opening hours

58. Published Configuration

Recommended conceptual object:

PublishedMapConfig

map
theme
languages
categories
menu
places

This object is read-only after publishing.

59. Versioning

Example:

Draft Version: 18
Published Version: 17

Client publishes:

Published Version: 18

Android:

Local Version: 17
Remote Version: 18
↓
Download

60. Stage 10 — Offline Cache

Android caches:

Published configuration
Place data
Translations
Menu
Theme
Marker metadata
Images

Recommended local database:

Room

Image cache:

Coil

61. Offline Startup

Android Starts



Read Local Cache



Show Tourist Map Immediately



Background Remote Version Check

Do not show a blank screen while waiting for Firebase.

62. Failed Update Rule

If new configuration download fails:

Continue using last valid published version

Never replace valid local data with incomplete data.

63. Stage 1P — Device Registration

Each signage device has:

deviceId
customerId
mapId
deviceName

Example:

deviceId:
dev_kyoto_entrance_01

64. Device Assignment

Stage 1 can initially use a device setup screen protected by PIN.

Example:

Device Setup

Customer ID
Map ID
Device Name

[Activate]

Better future implementation:

65. Device Record

devices/{deviceId}

deviceId
customerId
mapId

name

status

appVersion
publishedMapVersion

lastSeen
lastSync

createdAt
updatedAt

66. Device Heartbeat

Android periodically updates:

lastSeen
appVersion
mapVersion
networkStatus

Recommended interval:

60 seconds

67. Stage 1Q — Kiosk Mode

Android signage should support:

Fullscreen immersive mode
Hide status bar
Hide navigation
Prevent accidental exit
Auto-start after reboot
Maintain screen awake
Recover after app crash
Reconnect after network failure

68. Idle Reset

Important for public signage.

Example:

```
idleTimeoutSeconds = 90
```

After timeout:

```
Stop Audio  
Close QR  
Close Detail Panel  
Clear Search  
Reset Category  
Reset Language to default  
Return Map Camera
```

Client can later configure timeout.

69. Stage 1 Firestore Structure

```
customers/  
  {customerId}  
  
users/  
  {uid}  
  
maps/  
  {mapId}  
  
maps/{mapId}/places/  
  {placeId}  
  
maps/{mapId}/categories/  
  {categoryId}  
  
maps/{mapId}/menu/  
  {menuItemId}  
  
maps/{mapId}/languages/  
  {languageId}  
  
devices/  
  {deviceId}  
  
publishedMaps/  
  {mapId}  
  
publishedMaps/{mapId}/versions/  
  {versionId}
```

70. Firebase Storage Structure

```
customers/  
  {customerId}/  
    branding/
```

```
maps/  
  {mapId}/  
    places/  
    markers/  
    gallery/
```

Example:

```
maps/map_kyoto_001/places/place_yodobashi/main.jpg
```

71. Security Rules

Client can access only:

```
resources where customerId == authenticated user's customerId
```

Android signage:

```
read published configuration
```

Android must not write Client CMS data.

Device heartbeat should have its own controlled write path.

72. Client API Boundary

Avoid Android depending directly on arbitrary Firestore structure.

Create a stable repository/domain layer.

Concept:

```
PublishedMapRepository
```

Functions:

```
getCurrentVersion(mapId)  
getPublishedMap(mapId, version)
```

This allows Firestore implementation to change later.

73. Android Architecture

Recommended:

```
UI
↓
ViewModel
↓
Use Cases
↓
Repository
↓
Remote / Local Data Sources
```

Example:

```
TouristMapScreen
↓
TouristMapViewModel
↓
ObservePublishedMapUseCase
↓
PublishedMapRepository
├─ Firebase
└─ Room
```

74. Android Feature Modules

Recommended:

app

core-model
core-data
core-ui
core-map
core-network

feature-map
feature-place-detail
feature-search
feature-language
feature-voice
feature-qr
feature-device

For a smaller first repository, these can initially be packages rather than Gradle modules.

75. Client Admin Routes

Recommended:

/register
/login

```
/admin
/admin/map
/admin/places
/admin/places/new
/admin/places/{placeId}
/admin/categories
/admin/languages
/admin/menu
/admin/appearance
/admin/preview
/admin/publish
/admin/account
```

76. Mobile Web Routes

```
/m/{mapId}/p/{placeId}
```

Optional language query:

```
?lang=en
```

Example:

```
/m/map_kyoto_001/p/place_yodobashi?lang=en
```

77. Stage 1 Error Handling

Client Admin must clearly handle:

```
Firebase unavailable
Upload failure
Google Places error
Invalid coordinates
Publish validation error
Authentication expired
Permission denied
```

Android must handle:

```
No internet
Invalid published configuration
Image loading failure
Map initialization failure
TTS failure
QR generation failure
```

78. Stage 1 Logging

Client web:

Authentication events
Publish attempts
Publish failures
Content update failures

Android:

Startup
Device activation
Map sync
Map version change
Cache load
Map rendering failure
TTS failure
QR failure

Do not log sensitive credentials.

79. Stage 1 Analytics Foundation

Full analytics can come later.

But define Stage 1 event schema now.

Possible events:

PLACE_OPENED
VOICE_PLAYED
QR_OPENED
CATEGORY_SELECTED
SEARCH_USED
LANGUAGE_CHANGED

Initially these can be disabled or minimally collected.

80. Stage 1 Development Order

Phase 1A

Foundation

Repository
Firebase
Next.js
Android
Authentication
Multi-tenant model

Phase 1B

Client CMS Core

Dashboard
Map settings
Categories

Phase 1C

Places

Custom places
Google Places import
Images
Coordinates

Phase 1D

Languages

Language settings
Place translations
Fallback

Phase 1E

Android Map

Google Map
Published places
Photo markers
Category filters
Bottom menu

Phase 1F

Place Detail

Side panel
Images
Description
Address
Opening information

Phase 1G

Voice

TTS

Play / stop

Language integration

Phase 1H

QR

QR generation

Mobile destination page

Google Maps directions

Phase 1I

Appearance

Logo

Theme

Menu customization

Phase 1J

Preview / Publish

Draft

Preview

Validation

Versioned publish

Phase 1K

Android Sync / Offline

Room cache

Published version sync

Offline startup

Failed-update protection

Phase 1L

Production Signage

Device registration
Heartbeat
Kiosk mode
Idle reset
Crash recovery

81. First Development Milestone

Before designing all Android screens, first milestone should be:

Client registers
↓
customerId created
↓
mapId created
↓
Client logs in
↓
Adds one custom place
↓
Presses Publish
↓
Android reads published data
↓
Marker appears

This validates the complete architecture.

82. Second Milestone

Add 10 Places
↓
Photo Markers
↓
Category Filters
↓
Marker Selection
↓
Left Detail Panel

83. Third Milestone

English
Japanese
Chinese

↓
Tourist changes language
↓
Map content changes
↓
Audio guide reads selected language

84. Fourth Milestone

Select Yodobashi
↓
Show QR
↓
Scan phone
↓
Mobile page
↓
Open Google Maps
↓
Current location → Yodobashi

85. Fifth Milestone

Client edits content
↓
Android remains on old version
↓
Client previews
↓
Client publishes
↓
Android detects new version
↓
Updates safely

This proves the production publish architecture.

86. Stage 1 Demo Data

Recommended first demo:

Map:
Kyoto Station Tourist Guide

Categories:

Sightseeing
Gourmet
Shopping
Transport

Languages:

English
Japanese
Chinese

Places:

Kyoto Tower
Yodobashi Camera Kyoto
Kyoto Station
Hotel Granvia Kyoto
Porta
Restaurants
Temple / sightseeing locations
Bus information location

Use approximately:

10-20 places

87. Stage 1 Final Acceptance Test

Stage 1 passes when all of the following work on production-like infrastructure.

Client creates account
↓
Client receives own customerId
↓
Client receives own mapId
↓
Client configures map
↓
Client creates categories
↓
Client adds Google Place
↓
Client adds custom place
↓
Client adds images
↓
Client creates multilingual descriptions
↓
Client configures menu
↓
Client previews
↓
Client publishes
↓
Android signage syncs configuration
↓
Photo markers render correctly
↓
Tourist filters category
↓
Tourist selects marker
↓
Left side panel opens
↓
Correct translated description appears
↓
Voice guide speaks
↓
QR appears
↓
Phone successfully scans QR
↓
Mobile destination page opens
↓
Google Maps directions open

↓
Android resets after inactivity
↓
Internet disconnect test
↓
Cached map continues working
↓
Internet returns
↓
Android sync resumes

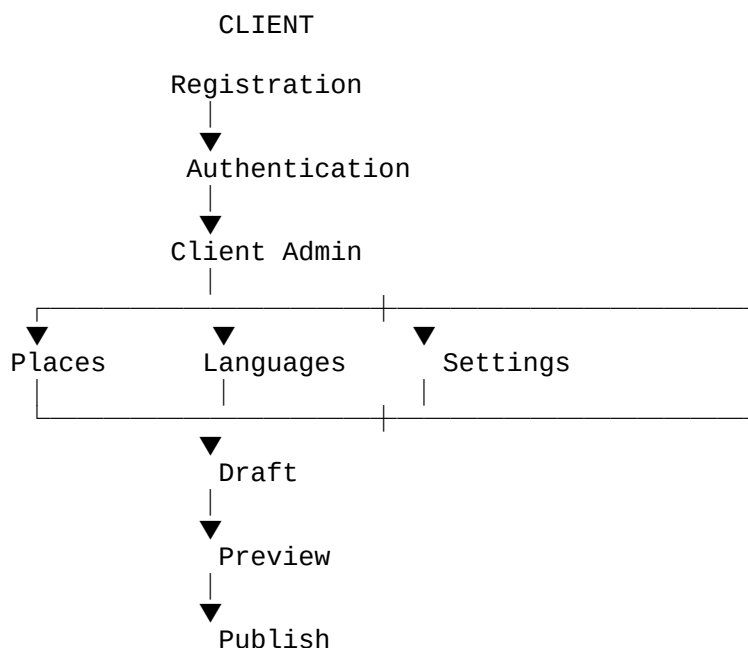
88. Stage 1 Out of Scope

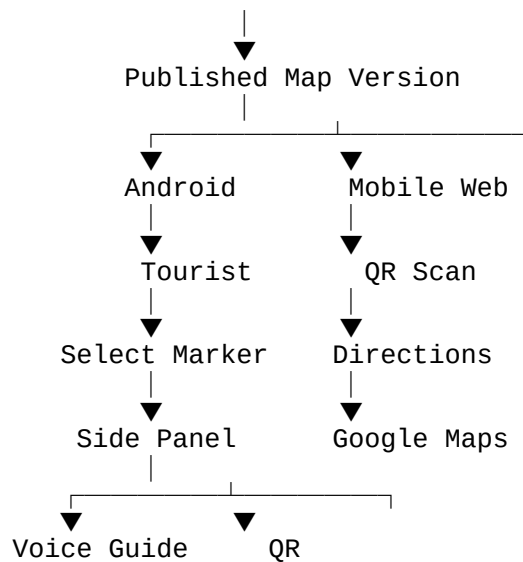
Do not allow these to delay Stage 1:

Super Admin Store
Paid subscriptions
Advanced billing
Weather widgets
Currency widgets
Complex drag/drop widget system
Advanced analytics dashboard
AI automatic translation
AI generated tour guide
WebRTC live cameras
Train APIs
Emergency APIs
Advertisement platform
Advanced Store themes
Full remote Android MDM

The architecture can prepare for them, but they belong to later stages.

89. Stage 1 Final Architecture





90. Stage 1 Product Definition

Stage 1 is not simply an Android map.

It is the first complete version of:

Client Tourism CMS
+
Publishing Platform
+
Android Tourist Signage
+
Multilingual Voice Guide
+
QR Smartphone Navigation

The most important engineering principle is:

Client edits DRAFT content.

Tourists only see PUBLISHED content.

And:

Android always retains the last valid published configuration.

These two rules should remain fundamental throughout the entire project.